POKEMON FANGAME

Documentation de la Base de Données

Architecture Supabase & Synchronisation IndexedDB

Document de référence pour Claude Code
Mars 2026

1. Vue d'ensemble de la base de données

La base de données Supabase (PostgreSQL) contient 9 tables organisées autour de 3 domaines fonctionnels : le référentiel Pokémon (pokemon, moves, pokemon_learnset, items), le monde du jeu (zones, wild_encounters, gyms), et les dresseurs (trainers, trainers_kanto). Toutes les données sont en lecture seule côté joueur — elles décrivent le contenu statique du jeu.

Chaque table utilise un identifiant textuel en slug (ex. bulbasaur, route-1, pewter-gym) sauf les tables de jonction qui utilisent des clés numériques. Les colonnes contenant du JSON stocké en texte (types, base_stats, team, evolutions, npcs, etc.) doivent être parsées côté client.

1.1 Table « pokemon »

Contient les 150 Pokémon de la génération 1. Chaque ligne porte les stats de base, les types, les sprites, et les conditions d'évolution.

Colonne	Type	Description
id	int (PK)	Numéro du Pokédex national (1–150)
name	text	Nom anglais (Bulbasaur, Charmander…)
generation	int	Génération d'origine (toujours 1)
types	jsonb	Tableau de types : ["grass","poison"]
base_stats	jsonb	Objet {hp, attack, defense, spAtk, spDef, speed}
catch_rate	int	Taux de capture (0–255)
exp_group	text	Courbe d'XP (mediumslow, fast, etc.)
base_exp	int	XP donnée quand vaincu
ev_yield	jsonb	EV accordés au vainqueur : {spAtk: 1}
tm_learnset	jsonb	Tableau d'IDs de CT/CS compatibles
sprite_url	text	URL du sprite normal (PokeAPI)
shiny_sprite_url	text	URL du sprite shiny
evolutions	jsonb	Tableau d'objets {to, level, method, condition}

Relations : pokemon.id ←→ pokemon_learnset.pokemon_id, wild_encounters.pokemon_id, trainers.team[].pokemonId

1.2 Table « moves »

Référentiel des attaques. Chaque attaque possède un type, une catégorie (physical/special/status), une puissance, une précision, et un nombre de PP.

Colonne	Type	Description
id	int (PK)	Identifiant numérique de l'attaque
name	text	Nom de l'attaque (ex. Tackle, Thunderbolt)
type	text	Type élémentaire (fire, water, etc.)
category	text	physical | special | status

power	int | null	Puissance de base (null pour les attaques status)
accuracy	int | null	Précision en % (null = ne rate jamais)
pp	int	Points de pouvoir de base
priority	int	Priorité d'attaque (-7 à +5)
effect	jsonb	Effets spéciaux (stat changes, status, etc.)
description	text	Description textuelle en français

Relations : moves.id ←→ pokemon_learnset.move_id, trainers.team[].moves[]

1.3 Table « pokemon_learnset »

Table de jonction many-to-many entre pokemon et moves. Définit quelles attaques chaque Pokémon apprend par montée de niveau. Environ 2 190 entrées.

Colonne	Type	Description
pokemon_id	int (FK)	Référence pokemon.id
move_id	int (FK)	Référence moves.id
level	int	Niveau auquel l'attaque est apprise (1 = de base)

Clé primaire composite : (pokemon_id, move_id)

1.4 Table « items »

Catalogue complet des objets du jeu (~2 100 entrées). Inclut les potions, les balls, les CT/CS, les objets tenus, et les objets clés.

Colonne	Type	Description
id	text (PK)	Slug unique (potion, master-ball, tm-15...)
name	text	Nom en français (Potion, Master Ball, CT15...)
description	text	Description en français
category	text	Catégorie (pokeballs, medicine, vitamins, held-items...)
price	int	Prix d'achat en Pokédollars
usable_in_battle	bool	Utilisable en combat
usable_outside	bool	Utilisable hors combat
effect	jsonb	Objet décrivant l'effet {type, target, value...}
sprite	text | null	URL du sprite (PokeAPI)

1.5 Table « zones »

Définit les 46 zones du monde (villes, routes, grottes, bâtiments). Chaque zone contient ses connexions, conditions de déblocage, et ses NPCs avec leurs dialogues et items.

Colonne	Type	Description
id	text (PK)	Slug unique (route-1, argenta, mt-moon...)
type	text	city | route | cave | building | special
name	text	Nom affiché en français
region	text	Région (kanto)
generation	int	Génération du contenu
has_shop	bool	La zone a un magasin
has_center	bool	La zone a un Centre Pokémon
gym_id	text (FK) | null	Référence gyms.id si la zone contient une arène
connected_zones	jsonb	Tableau de zone IDs accessibles
unlock_condition	jsonb	Condition pour accéder {type, zones}
npcs	jsonb	Tableau de NPCs : [{id, name, dialogue[], givesItem?, setsEvent?, requiredEvent?}]

Structure NPC imbriquée : chaque NPC possède un id unique, un nom, un tableau de dialogues, et optionnellement des champs givesItem (slug d'item), setsEvent et requiredEvent pour le système de quêtes.

1.6 Table « wild_encounters »

Définit les Pokémon sauvages rencontrables dans chaque zone, avec leurs niveaux et taux d'apparition. ~148 entrées.

Colonne	Type	Description
id	int (PK)	Identifiant auto-incrémenté
zone_id	text (FK)	Référence zones.id
pokemon_id	int (FK)	Référence pokemon.id
min_level	int	Niveau minimum de rencontre
max_level	int	Niveau maximum de rencontre
rate	int	Probabilité d'apparition (somme = 100 par zone/type)
encounter_type	text	grass | water | cave | fishing

1.7 Table « gyms »

Les 8 arènes de Kanto + la Ligue. Chaque arène a un champion, un type spécialisé, un badge, et un ordre dans la progression.

Colonne	Type	Description
id	text (PK)	Slug unique (pewter-gym, cerulean-gym...)
name	text	Nom de l'arène en français
leader	text	Nom du champion
type	text	Type spécialisé (rock, water, electric...)

badge	text	Nom du badge obtenu
order	int	Ordre dans la progression (1–8)
zone_id	text (FK)	Zone où se situe l'arène

1.8 Tables « trainers » & « trainers_kanto »

Deux tables avec le même schéma. trainers contient les dresseurs spéciaux (champions d'arène, Conseil des 4, rival) tandis que trainers_kanto contient les ~350 dresseurs normaux placés sur les routes et dans les grottes.

Colonne	Type	Description
id	text (PK)	Slug unique (youngster-1, agatha…)
name	text	Nom affiché
trainer_class	text	Classe (Youngster, Beauty, Lass…)
reward	int	Récompense en Pokédollars
zone_id	text (FK)	Zone où se trouve le dresseur
team	jsonb	Tableau d'objets : [{pokemonId, level, moves: [int]}]

Note importante : team[].pokemonId référence pokemon.id et team[].moves[] référence moves.id. Ces relations sont implicites (stockées en JSON, pas de FK PostgreSQL).

2. Carte des relations

Le schéma suit un modèle en étoile autour de trois entités centrales : pokemon, moves, et zones. Voici les liens entre toutes les tables :

De	Vers	Type de lien	Via
pokemon_learnset	pokemon	FK	pokemon_learnset.pokemon_id → pokemon.id
pokemon_learnset	moves	FK	pokemon_learnset.move_id → moves.id
wild_encounters	pokemon	FK	wild_encounters.pokemon_id → pokemon.id
wild_encounters	zones	FK	wild_encounters.zone_id → zones.id
trainers	zones	FK	trainers.zone_id → zones.id
trainers.team[]	pokemon	JSON	team[].pokemonId → pokemon.id
trainers.team[]	moves	JSON	team[].moves[] → moves.id
zones	gyms	FK	zones.gym_id → gyms.id
zones.npcs[]	items	JSON	npcs[].givesItem → items.id
pokemon	items (CT)	JSON	pokemon.tm_learnset[] → moves.id

3. Architecture de synchronisation Supabase → IndexedDB

3.1 Pourquoi une IndexedDB locale ?

- Performance : éviter les requêtes réseau à chaque rencontre sauvage, chaque combat, chaque consultation du Pokédex.
- Mode hors-ligne : le joueur peut jouer sans connexion une fois les données téléchargées.
- Données du joueur : la sauvegarde locale (inventaire, équipe, progression) vit uniquement dans IndexedDB.
- Réduction des coûts Supabase : une seule lecture initiale au lieu de milliers de petites requêtes.

3.2 Principe général

Le système sépare les données en deux catégories :

Catégorie	Source de vérité	Tables concernées
Game Data (statique)	Supabase	pokemon, moves, pokemon_learnset, items, zones, wild_encounters, gyms, trainers, trainers_kanto
Player Data (dynamique)	IndexedDB	player_team, player_inventory, player_progress, player_pokedex

Le Game Data est téléchargé depuis Supabase au premier lancement, puis mis en cache dans IndexedDB. Le Player Data est créé et modifié localement et n'existe que dans IndexedDB (pas de sauvegarde cloud — à ajouter ultérieurement si nécessaire).

3.3 Schéma IndexedDB

On utilise une seule base nommée pokemon_game_db. Voici les object stores :

Object Store	keyPath	Index	Source
pokemon	id	—	Supabase
moves	id	—	Supabase
pokemon_learnset	auto	pokemon_id, move_id	Supabase
items	id	category	Supabase
zones	id	region	Supabase
wild_encounters	id	zone_id	Supabase
gyms	id	order	Supabase
trainers	id	zone_id	Supabase
trainers_kanto	id	zone_id	Supabase
—	—	—	—
player_team	slot	—	Local
player_inventory	item_id	category	Local

player_progress	key	—	Local
player_pokedex	pokemon_id	—	Local

3.4 Implémentation : module db.ts

Voici le squelette du module de synchronisation en TypeScript. Il utilise idb (wrapper Promise autour d'IndexedDB) et le client Supabase.

```
// db.ts
import { openDB, DBSchema, IDBPDatabase } from 'idb';
import { supabase } from './supabaseClient';

const DB_NAME = 'pokemon_game_db';
const DB_VERSION = 1;
const SYNC_KEY = 'last_sync_timestamp';

// ── 1. Définition du schéma TypeScript ──

interface GameDB extends DBSchema {
  pokemon:         { key: number; value: Pokemon; };
  moves:           { key: number; value: Move; };
  pokemon_learnset: { key: number; value: Learnset;
                    indexes: { 'pokemon_id': number }; };
  items:           { key: string; value: Item;
                    indexes: { 'category': string }; };
  zones:           { key: string; value: Zone;
                    indexes: { 'region': string }; };
  wild_encounters:  { key: number; value: WildEncounter;
                    indexes: { 'zone_id': string }; };
  gyms:            { key: string; value: Gym; };
  trainers:        { key: string; value: Trainer;
                    indexes: { 'zone_id': string }; };
  trainers_kanto:   { key: string; value: Trainer;
                    indexes: { 'zone_id': string }; };
  // Player stores
  player_team:     { key: number; value: PlayerPokemon; };
  player_inventory: { key: string; value: PlayerItem;
                    indexes: { 'category': string }; };
  player_progress:  { key: string; value: ProgressEntry; };
  player_pokedex:   { key: number; value: PokedexEntry; };
}


// ── 2. Ouverture de la base ──

export async function getDB(): Promise<IDBPDatabase<GameDB>> {
  return openDB<GameDB>(DB_NAME, DB_VERSION, {
    upgrade(db) {
      // Game Data stores
      db.createObjectStore('pokemon', { keyPath: 'id' });
      db.createObjectStore('moves', { keyPath: 'id' });
      const ls = db.createObjectStore('pokemon_learnset',
        { autoIncrement: true });
      ls.createIndex('pokemon_id', 'pokemon_id');
      const items = db.createObjectStore('items',
        { keyPath: 'id' });
      items.createIndex('category', 'category');
      const zones = db.createObjectStore('zones',
        { keyPath: 'id' });
      zones.createIndex('region', 'region');
      const we = db.createObjectStore('wild_encounters',
        { keyPath: 'id' });
      we.createIndex('zone_id', 'zone_id');
```

```
      db.createObjectStore('gyms', { keyPath: 'id' });
      const tr = db.createObjectStore('trainers',
        { keyPath: 'id' });
      tr.createIndex('zone_id', 'zone_id');
      const tk = db.createObjectStore('trainers_kanto',
        { keyPath: 'id' });
      tk.createIndex('zone_id', 'zone_id');
      // Player Data stores
      db.createObjectStore('player_team', { keyPath: 'slot' });
      const pi = db.createObjectStore('player_inventory',
        { keyPath: 'item_id' });
      pi.createIndex('category', 'category');
      db.createObjectStore('player_progress',{ keyPath:'key'});
      db.createObjectStore('player_pokedex',
        { keyPath: 'pokemon_id' });
    }
  });
}


// ── 3. Synchronisation depuis Supabase ──

const GAME_TABLES = [
  'pokemon', 'moves', 'pokemon_learnset', 'items',
  'zones', 'wild_encounters', 'gyms',
  'trainers', 'trainers_kanto'
] as const;

export async function syncFromSupabase(
  onProgress?: (table: string, pct: number) => void
): Promise<void> {
  const db = await getDB();
  const total = GAME_TABLES.length;

  for (let i = 0; i < total; i++) {
    const table = GAME_TABLES[i];
    onProgress?.(table, ((i) / total) * 100);

    // Fetch tout le contenu de la table
    const { data, error } = await supabase
      .from(table).select('*');
    if (error) throw new Error(
      `Sync failed for ${table}: ${error.message}`);

    // Parse les colonnes JSON si nécessaire
    const parsed = data.map(row => parseJsonCols(row));

    // Écriture en transaction unique par table
    const tx = db.transaction(table, 'readwrite');
    await tx.store.clear();
    for (const row of parsed) {
      await tx.store.put(row);
    }
    await tx.done;
  }

  // Marquer le timestamp de sync
  const pdb = db.transaction('player_progress', 'readwrite');
  await pdb.store.put({
    key: SYNC_KEY, value: Date.now()
  });
  await pdb.done;

  onProgress?.('done', 100);
}


// ── 4. Helpers JSON ──
```

```
const JSON_COLUMNS: Record<string, string[]> = {
  pokemon: ['types','base_stats','ev_yield','tm_learnset',
            'evolutions'],
  items: ['effect'],
  zones: ['connected_zones','unlock_condition','npcs'],
  trainers: ['team'],
  trainers_kanto: ['team'],
  moves: ['effect'],
};

function parseJsonCols(row: Record<string, any>) {
  const table = row._table; // ou logique contextuelle
  const cols = JSON_COLUMNS[table] ?? [];
  for (const col of cols) {
    if (typeof row[col] === 'string') {
      try { row[col] = JSON.parse(row[col]); }
      catch { /* already parsed or invalid */ }
    }
  }
  return row;
}
```

3.5 Vérification de fraîcheur & mise à jour

Au lancement de l'application, le système vérifie si une synchronisation complète a déjà eu lieu. Si non, il télécharge tout. Si oui, il peut optionnellement vérifier un numéro de version stocké côté serveur pour décider s'il faut re-synchroniser.

```
// ── 5. Logique de lancement ──

export async function initGameData(
  onProgress?: (t: string, p: number) => void
): Promise<void> {
  const db = await getDB();

  // Check si déjà sync
  const tx = db.transaction('player_progress', 'readonly');
  const syncEntry = await tx.store.get(SYNC_KEY);
  await tx.done;

  if (!syncEntry) {
    // Premier lancement : sync complète
    await syncFromSupabase(onProgress);
    return;
  }

  // Optionnel : vérifier la version distante
  const { data } = await supabase
    .from('game_meta')
    .select('value')
    .eq('key', 'data_version')
    .single();

  const local = await db.get(
    'player_progress', 'data_version');
  if (data?.value !== local?.value) {
    await syncFromSupabase(onProgress);
    await db.put('player_progress',
      { key: 'data_version', value: data?.value });
  }
}
```

3.6 Accès aux données en jeu

Une fois la sync effectuée, tout passe par IndexedDB. Voici les patterns de lecture les plus fréquents :

```
// ── 6. Exemples d'accès ──

// Récupérer un Pokémon par son ID
const bulbasaur = await db.get('pokemon', 1);

// Toutes les rencontres d'une zone
const encounters = await db.getAllFromIndex(
  'wild_encounters', 'zone_id', 'route-1');

// Tous les dresseurs d'une zone
const trainers = await db.getAllFromIndex(
  'trainers_kanto', 'zone_id', 'mt-moon');

// Learnset complet d'un Pokémon
const learnset = await db.getAllFromIndex(
  'pokemon_learnset', 'pokemon_id', 25);

// Items d'une catégorie
const balls = await db.getAllFromIndex(
  'items', 'category', 'pokeballs');

// Progression du joueur
const badges = await db.get(
  'player_progress', 'badges');
const team = await db.getAll('player_team');
```

3.7 Stores locaux du joueur

Ces stores n'existent que dans IndexedDB et ne sont jamais synchronisés depuis Supabase :

Store	keyPath	Contenu type
player_team	slot (0–5)	{ slot, pokemonId, nickname, level, currentHp, moves[], ivs, evs, status }
player_inventory	item_id	{ item_id, quantity, category }
player_progress	key	{ key, value } — badges, events, current_zone, money, play_time…
player_pokedex	pokemon_id	{ pokemon_id, seen: bool, caught: bool, caught_count }

3.8 Bonnes pratiques

1. Toujours utiliser des transactions pour les écritures batch (sync et sauvegarde).
2. Parser les colonnes JSON au moment de la sync, pas à chaque lecture.
3. Créer des index IndexedDB sur les colonnes utilisées pour filtrer (zone_id, pokemon_id, category).

4. Exposer un hook React useDB() qui gère l'ouverture de la connexion et la met en cache.
5. Afficher une barre de progression pendant la sync initiale (~2–3 secondes pour 9 tables).
6. Prévoir une fonction resetGameData() qui vide les stores player_* pour un « New Game ».
7. Ne jamais appeler Supabase en dehors du module de sync — tout le reste du jeu lit IndexedDB.